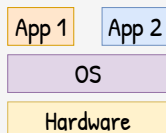
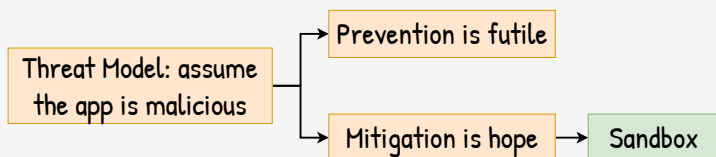
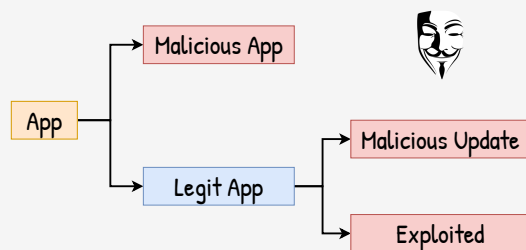


Sandbox at OS level: Seccomp, cgroups, capabilities, namespaces & mounts

Taller de Seguridad
Ing. Martin Di Paola - Fiuba



Que tan seguro es correr una app? Podria ser maliciosa. Podes revisar el código, usar analizadores estáticos, hacer un monitoreo dinámico (lease un antivirus), todo para cazar a una app maliciosa antes de que haga daño.

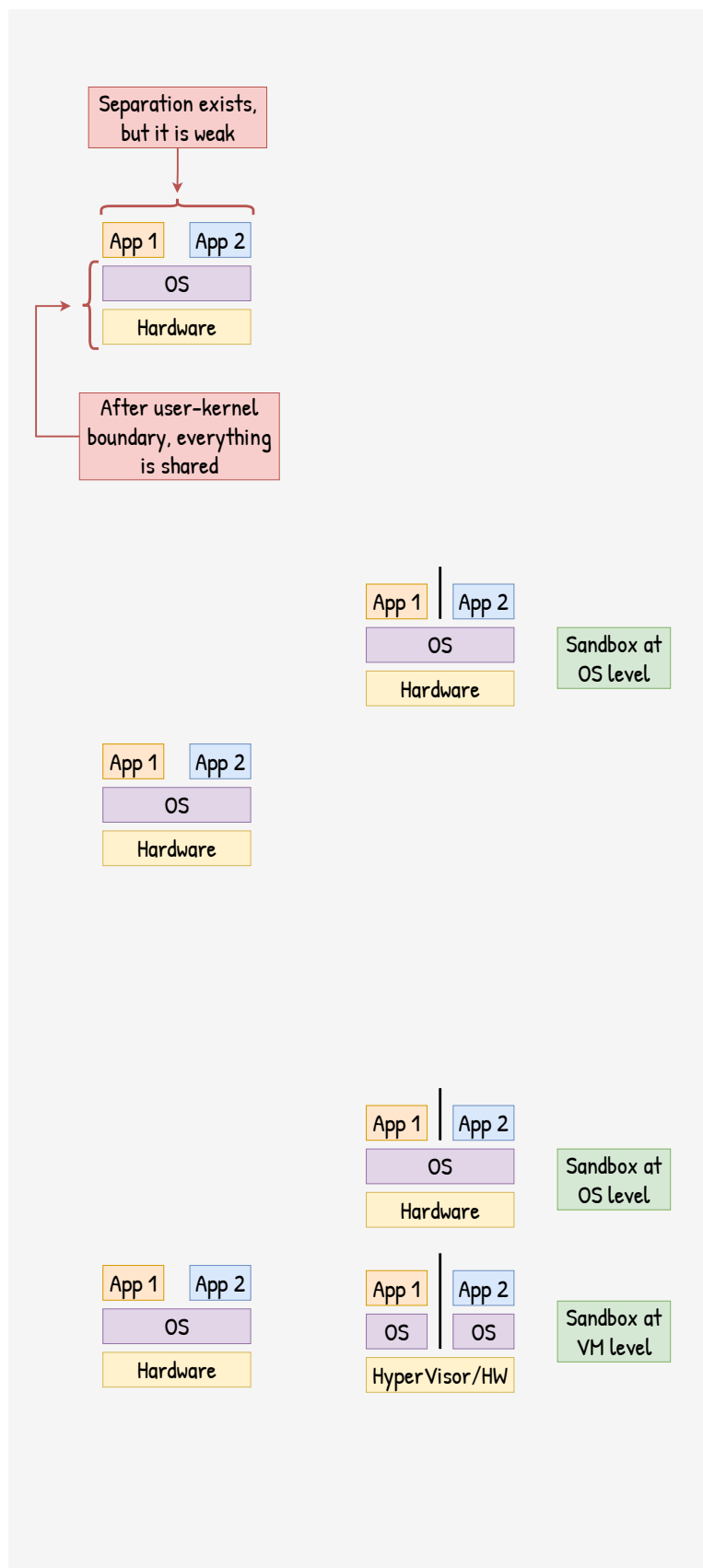
Pero no importa que hagas, algo se te puede escapar.

Incluso si la app es legit puede volverse maliciosa. Un bug y un input malicioso y la app se vuelve en tu contra. Y aun si sos cuidadoso, el desarrollador de la app lo es? El desarrollador de la lib que usa la lib que usa la lib que usa la lib, lo es?

Un descuido en algun lado y el próximo update de la app puede volverla maliciosa.

Prevenir es inutil. Un threat model más robusto es considerar que toda app es maliciosa y *mitigar* el daño, conteniendolo, limitandolo. A un sandbox.

Cual es el escanario del q partimos? Toda app que corre tiene acceso al OS y este al hardware.



En Linux una app que corre como un usuario no puede acceder a los archivos de otro usuario. Pero nada limita ver los files del mismo usuario.

Una calculadora maliciosa podría leer archivos secretos del usuario usados por otra app. Los límites entre apps son frágiles.

Y los archivos son uno de los muchos otros recursos que el kernel ofrece. Y todos están compartidos.

Es un mal punto de partida.

El objetivo es separar, aislar, poner a cada app en su propio sandbox para que no vean nada más de lo estrictamente necesario.

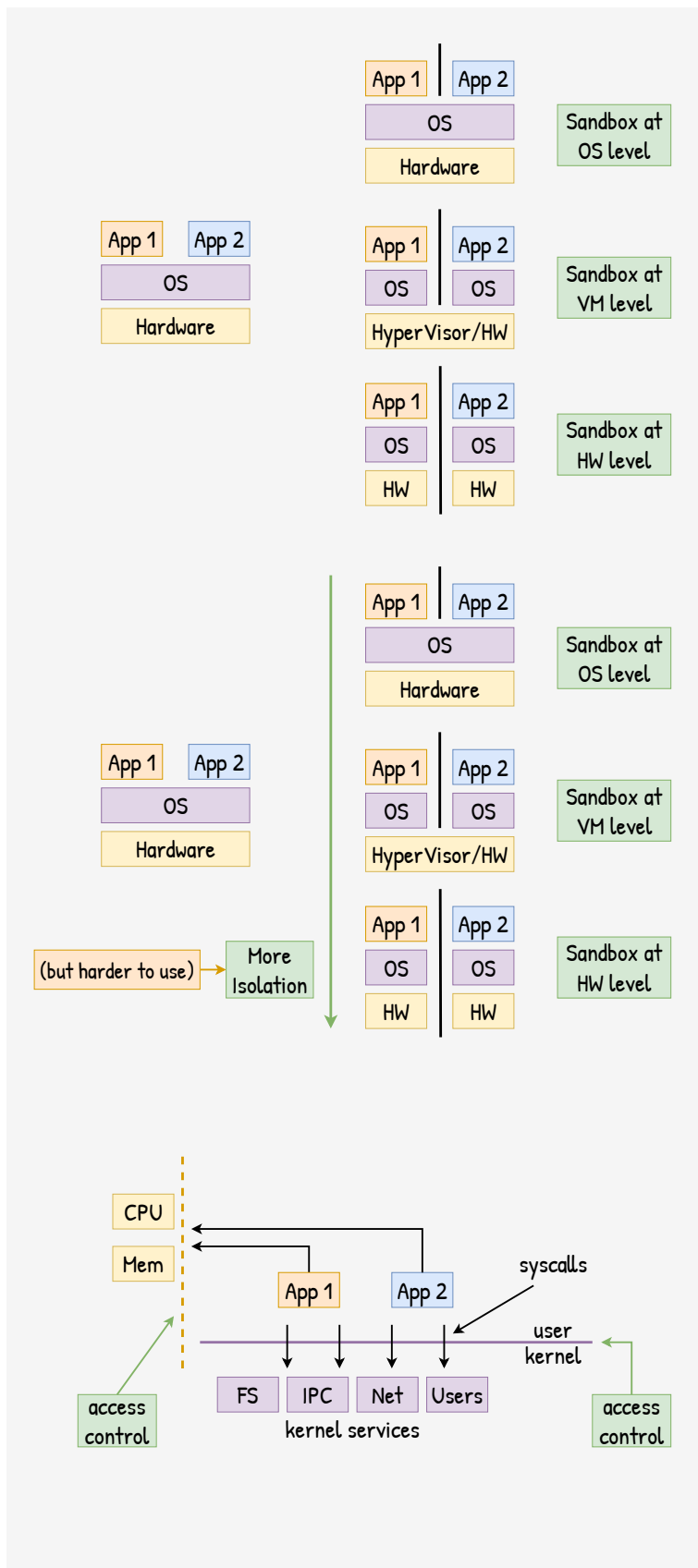
El primer enforcador es el OS. El administrador construye los sandboxes con los features que el OS le ofrezca y pone a las apps ahí adentro.

Toda la seguridad del sistema descansa en que no haya una mala configuración ni un bug en el kernel.

Pero los kernels tienen bugs. Una app que explote un bug puede escapar del sandbox e invadir otras áreas, sean otras apps o el propio kernel.

Los OSs son enormes y asumir que son bug-free es ingenuo. La alternativa es separar incluso al OS poniendo a cada app con su propio OS usando o unikernels o máquinas virtuales.

Lo único que comparten son el hypervisor y el hardware.



Y aunque los hypervisors son más chicos que un OS, y por ende son más fáciles de revisar, no estan libres de bugs.

Y aun se comparte el hardware. Una app podría, de alguna manera, interactuar con el hardware y escapar del sandbox, tomando control del hypervisor o leakeando información sensible de otra VM.

Para el paranoico, hay q tener hardwares separados: máquinas físicas separadas para apps distintas.

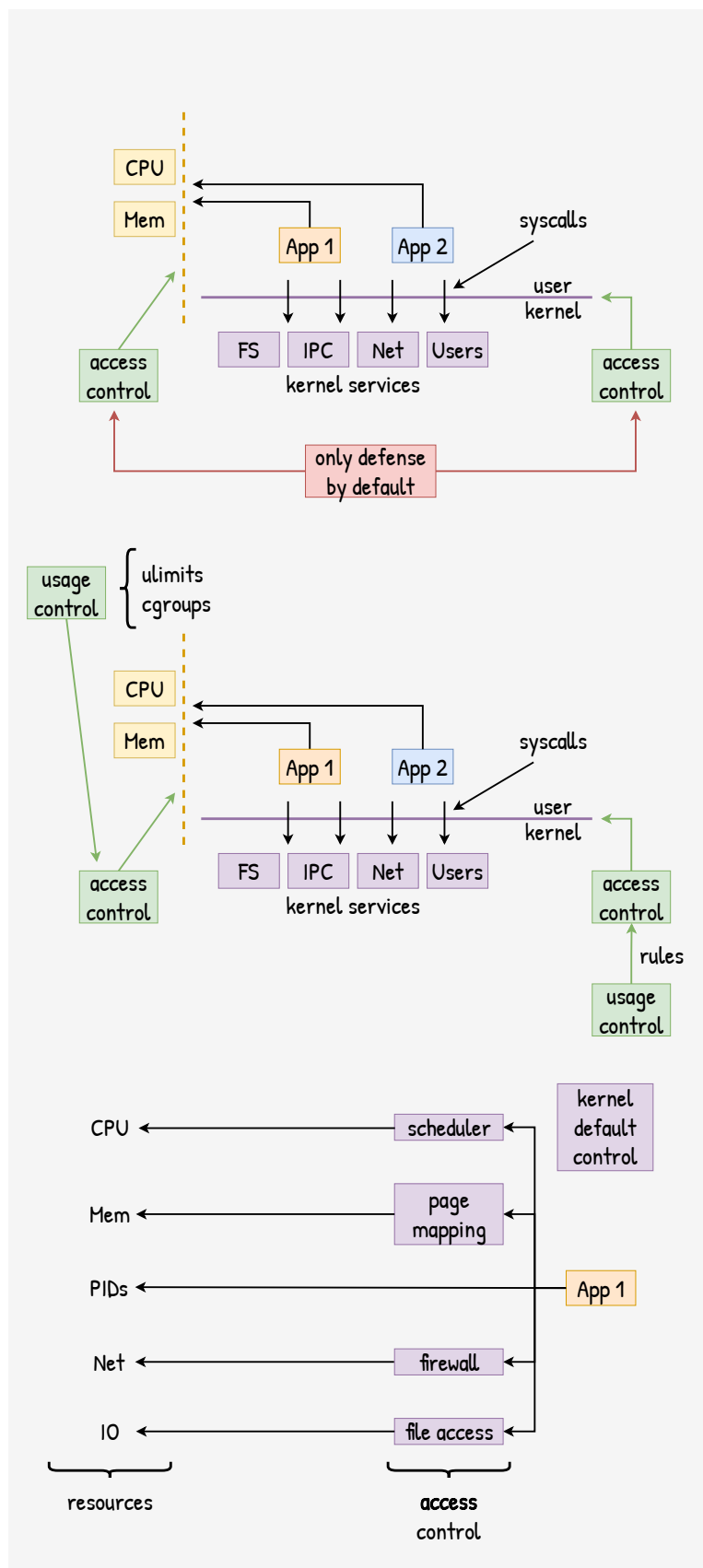
La filosofía de mitigación es simple: mayor aislamiento y separacion, mayor es la seguridad, aunque tambien mayor la dificultad de uso (lease, una peor UX)

Nos vamos a enfocar en construir sandboxes a nivel del OS (a nivel VM y HW vendra en clases posteriores). Entonces, como una app interactua con el sistema? en que puntos podemos limitarla?

La app puede usar la CPU y memoria directamente, aunque el kernel tiene puntos de control (ej: una app no puede ver la memoria de otra app)

File systems (FS), inter-process communications (IPC), networks (Net), users y más son servicios que ofrece el kernel, algunos como abstracciones del hardware (FS vs disco, Net vs placa de wifi), otros para soportar features convenientes (IPC, Users)

Las apps acceden a ellos via syscalls. Todas las apps puede ejecutar cualquier syscall, el kernel solo controla si se tiene permiso. Ej: una app puede ejecutar la syscall **open**, el kernel solo verifica si el user que corre la app puede abrir el archivo segun los permisos de él.



Por default, la única defensa son los controles de accesos que impone el kernel (y en Linux, esto está modelado como que puede o no hacer un user).

Para un esquema de múltiples usuarios, límites entre usuarios es razonable. Para múltiples apps corriendo para el mismo user, este modelo no ofrece seguridad alguna.

En Android, cada app corre con su propio usuario haciendo q estos controles tengan sentido (además de muchos más mecanismos de defensa q implementa el OS).

Un OS moderno ya ofrece control en la CPU y memoria: un proceso de user (no el kernel) no puede ejecutar instrucciones privilegiadas de la CPU, no puede acaparar el 100% de este y no puede acceder a memoria mapeada para otro proceso de user o del kernel.

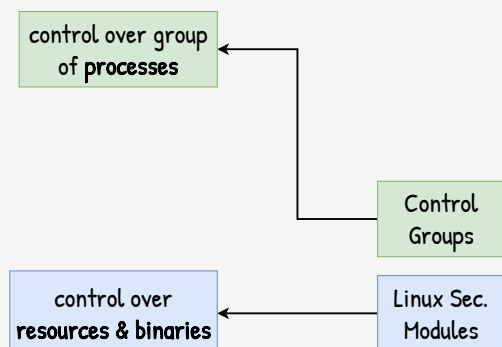
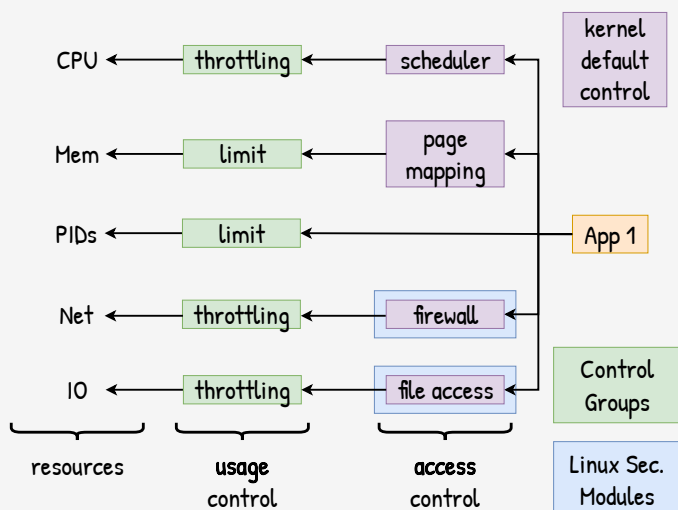
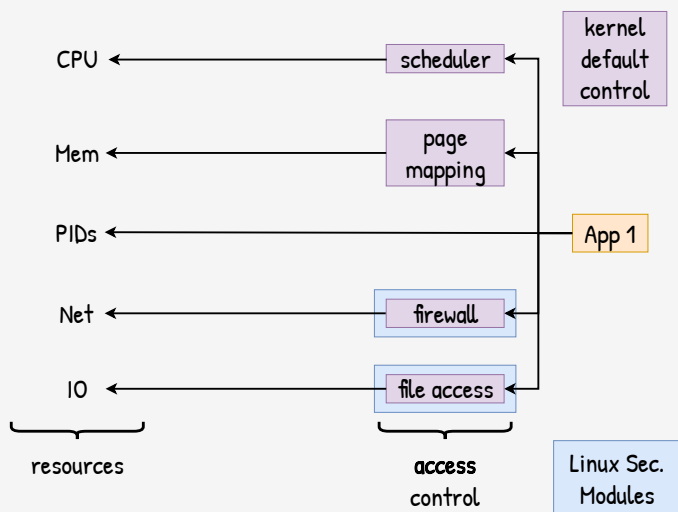
Pero por default no hay mayores límites. Una app bien podría consumir el 99.99% de la CPU o de la memoria. Así un sandbox también debe poner límites adicionales a estos recursos.

El kernel por default impone varios checks para controlar el acceso de una app a los recursos del sistema.

Es el scheduler quien dice cuando y por cuanto tiempo una app puede usar la CPU; es el memory system que dice que páginas de memoria le pertenecen a una app (y le impide que esta acceda a páginas q no son de ella).

Pero otros recursos están menos protegidos: una app puede crear tantos otros procesos como quieran (y consumir PIDs); acceso a la red está irrestricto salvo reglas de firewalls (que no son contra una app sino contra una IP/Puerto); o IO.

En Linux, por default, el acceso está controlado por los permisos que el user que corre la app tenga. En un sistema multi-user esto ofrece algo de aislamiento pero en un contexto de single-user corriendo múltiples apps, todas ellas pueden ver los files de las otras!

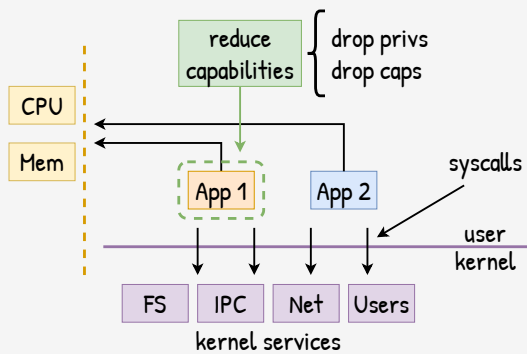


Linux Security Modules (LSMs) son modules que enfuerzan controles de acceso adicionales.

Apparmor, SELinux y TOMOYO son 3 LSMs con mecanismos tipo mandatory access control (MACs), esto es, el admin define que archivos puede o no acceder una app, aun si el user tiene acceso y si puede o no acceder a la red.

Control groups (cgroups) permiten tener un control fino sobre el uso de los recursos (asumiendo que los procesos tenga acceso en primer lugar).

Se puede limitar el uso de CPU, memoria, PIDs, network e IO.

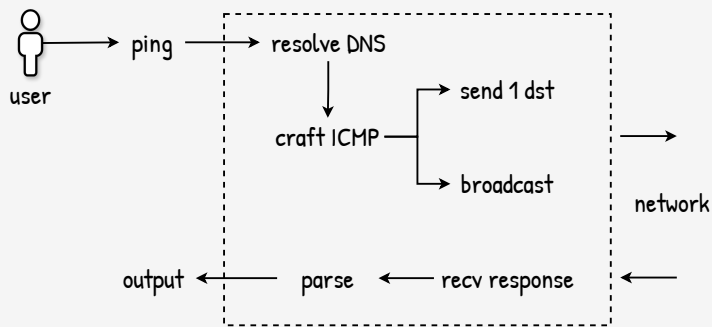


No existe el concepto de sandbox en el OS. Este se construye parte a parte usando múltiples tecnologías del OS.

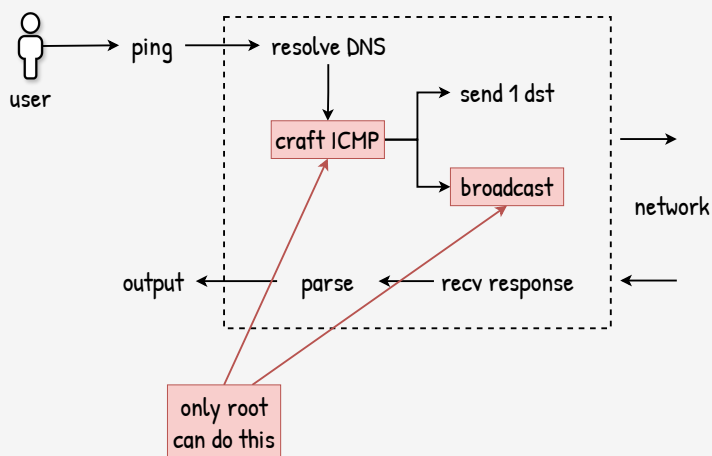
La primera es remover privilegios y capabilities de la app. Ciertas apps requieren de permisos especiales que solo **root** puede usar, pero concederles todos los poderes de **root** es exponerse demasiado.

Dichas apps debería ejecutar lo mínimo y necesario con permisos de root y luego remover todos los privilegios antes de continuar su ejecución, reduciendo así la exposición (lease un bug posterior tendrá control de la app pero no de una app privilegiada)

Desde el punto de vista de un sandbox, uno no debería confiar que la app sea honesta y remueva sus privilegios por motus proprio así que uno debe enforzarlo desde afuera.

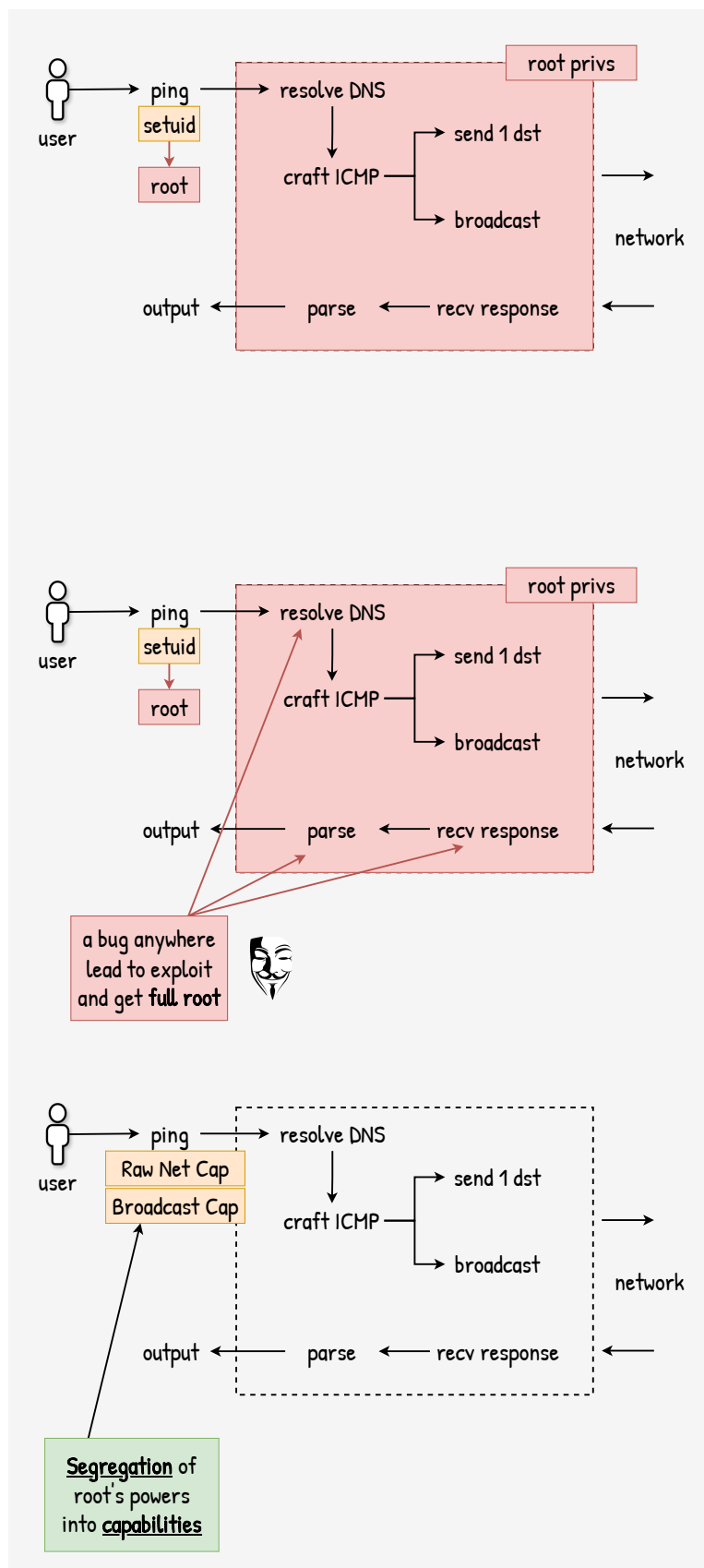


Veamos, la tool **ping**. El user puede enviar un mensaje ICMP tanto a un solo destino como a todo un rango haciendo un broadcast y por cada mensaje enviado, **ping** espera la respuesta (si es que llega) y muestra estadísticas sobre los tiempos de respuesta de vuelta al user.



Enviar mensajes ICMP requiere de privilegios de root. Solo TCP y UDP son los protocolos permitidos para las apps sin privilegios (sobre la red IP).

Además, hacer un broadcast también requiere de privilegios de root (uno no quiere que cualquier user esté saturando la red tan trivialmente, solo el administrador debería poder hacerlo).



Para que **ping** funcione debe correr como root pero no queremos q solo el administrador pueda hacer ping. Q los users del sistema puedan hacer un ping es sumamente práctico!

De alguna manera un user sin privilegios debe poder adquirir root para correr **ping**. Hay muchas formas pero la tradicional es que el administrador deje el bit **setuid** seteado para que se eleven los privilegios al momento de ejecutar la app.

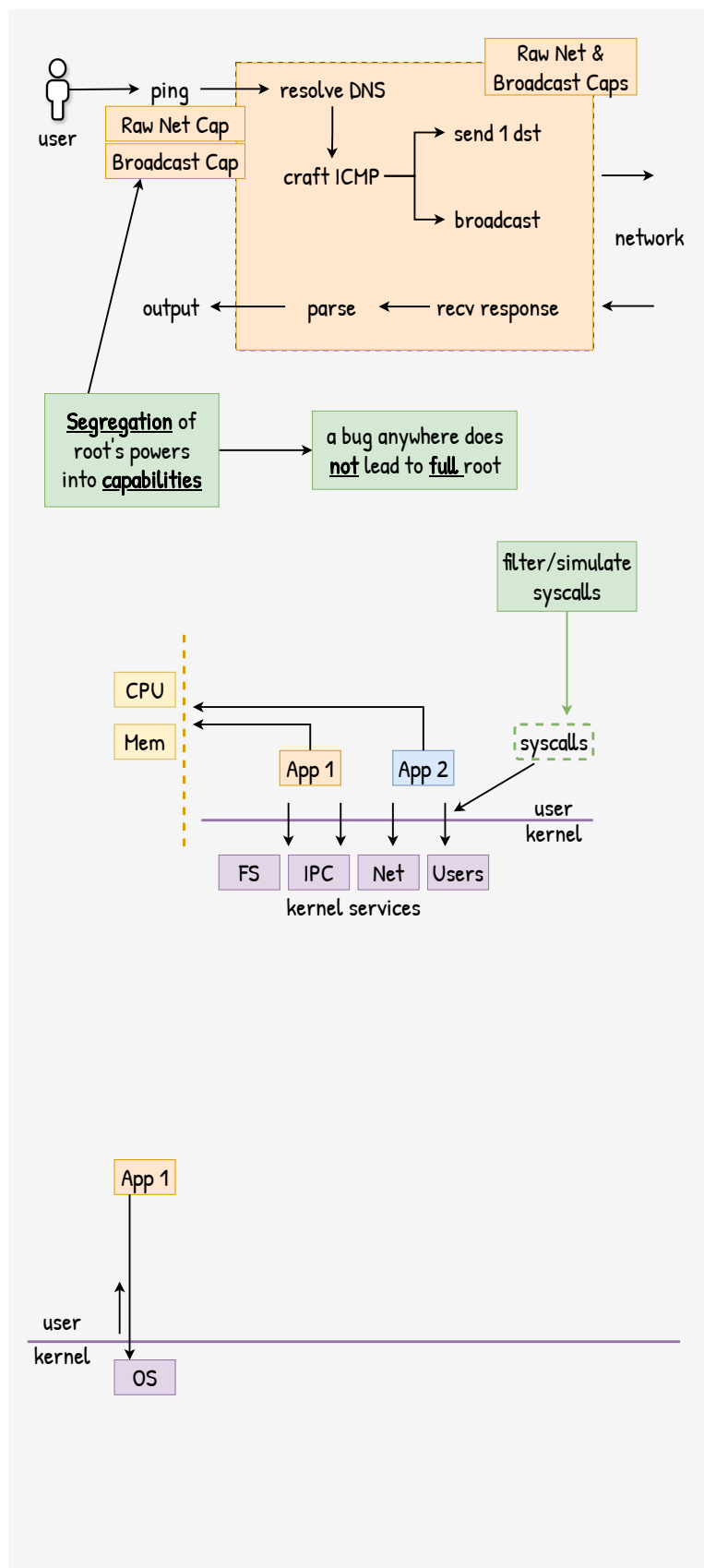
ping puede ser una app legit y, a pesar de correr como root, solo usa los mínimos recursos (ICMP vs tomar control de la maquina)

Pero toda la app corre como root, no solo aquella parte extrictamente necesaria. Un bug en cualquier lado y un adversario puede tomar control de la app y con ella, todo el poder de root.

Tradicionalmente solo existian users sin privilegios y root con todos los privilegios.

Pero desde ya hace un tiempo que estos privilegios se segregaron/partieron en sets más chicos llamados capabilities.

Root sigue teniendo todas las capabilities pero ahora el administrador puede attachar un subset de ellas a una app.



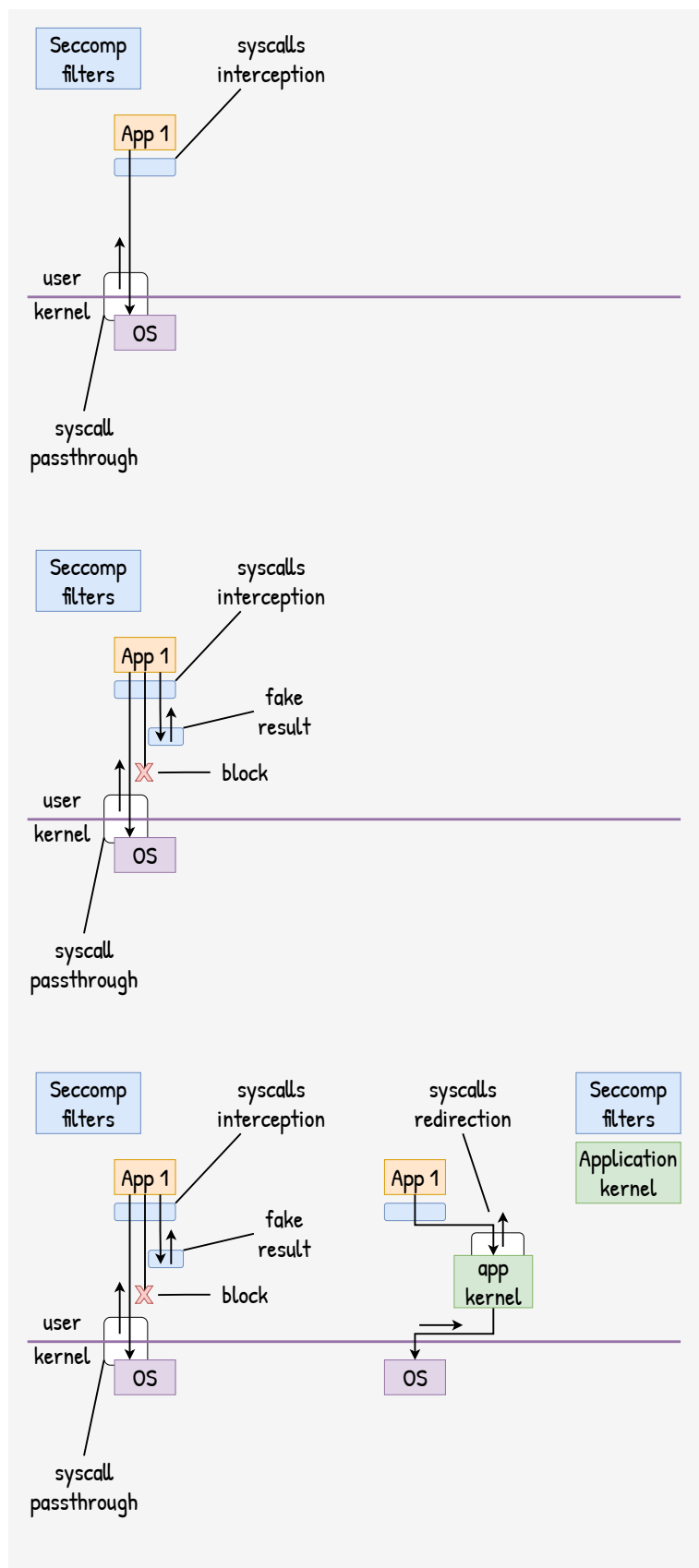
Al momento de ejecutarse **ping**, este adquiere solo las capabilities concedidas y las únicas necesarias para su funcionamiento (en este caso, poder armar mensajes ICMP y hacer broadcast)

Si hay un bug, un adversario puede tomar el control de la app y acceso a la máquina, pero no tendrá más capabilities que la dadas (es una situación mala, pero al menos no tiene todo el poder de fuego de root).

El siguiente nivel es cortar accesos al OS. El OS expone muchos servicios y formas de acceder a él: un bug y la app puede adquirir control del kernel.

Pero la gran mayoría de las apps no requieren de todos los servicios (para que una calculadora requiera hacer un **connect** ?). Bloquear las **syscalls** es reducir el área de ataque.

Una app interactúa con el kernel via syscalls: llamadas al sistema.



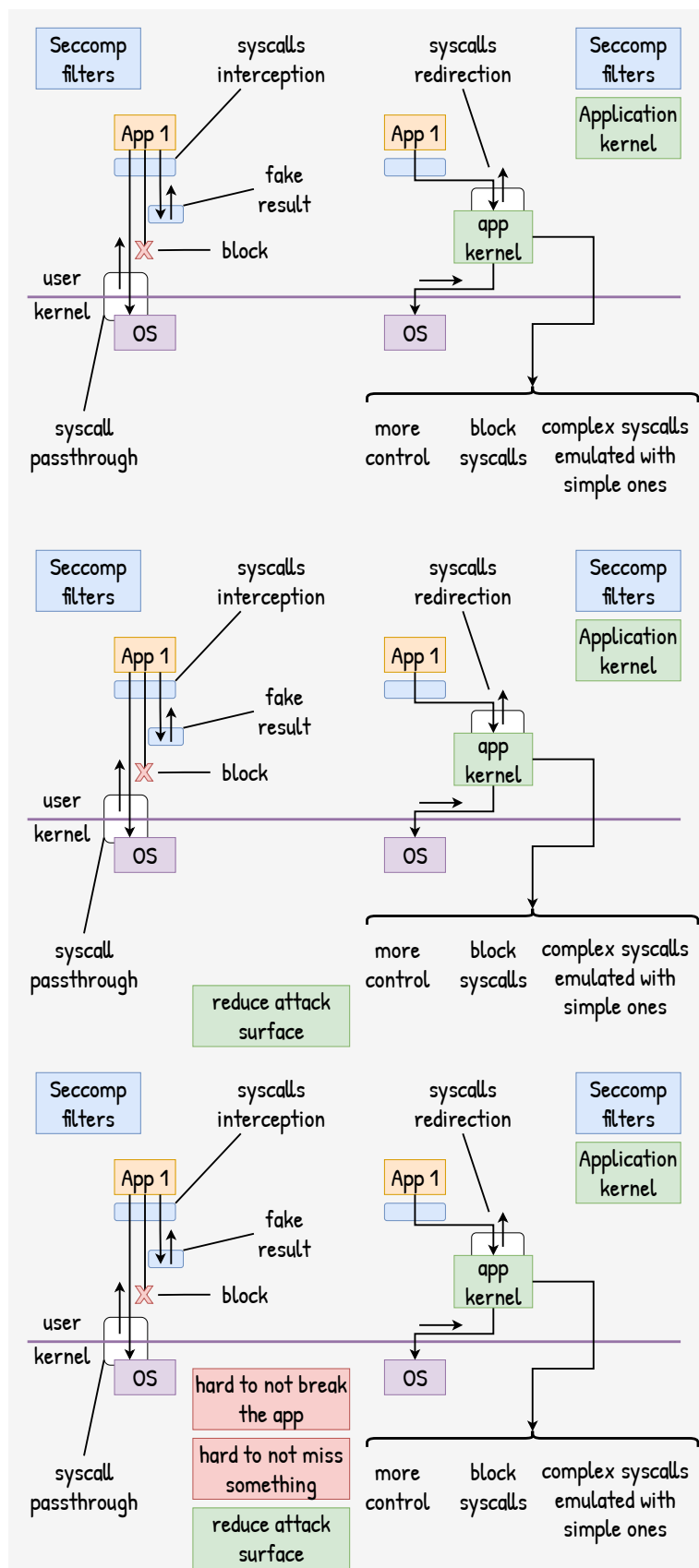
La magia esta en los filtros seccomp que permiten interceptar syscalls y (1) dejarlas pasar, (2) bloquearlas/fallarlas y (3) retornar valores falsos sin interactuar con el kernel realmente.

Esto permite hacer que una app no pueda interactuar de cierta manera con el kernel sin tener que modificar su código.

Un AppKernel es un filtro seccomp con esteroides. Un filtro seccomp que redirecciona todas las syscalls hacia una app que corre en userspace y esta o bien (1) emula el compute de las syscalls o (2) bloquea/falla.

Por que emular? Por que el kernel ofrece syscalls de implementación compleja que pueden ser reimplementadas (en userspace) con syscalls más simples, reduciendo la probabilidad de un bug en el kernel.

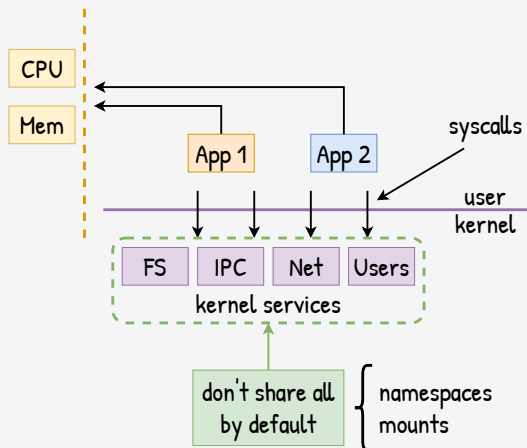
Y ademas, un appkernel permite hacer filtros mucho más configurables que los ofrecidos nativamente por los seccomp.



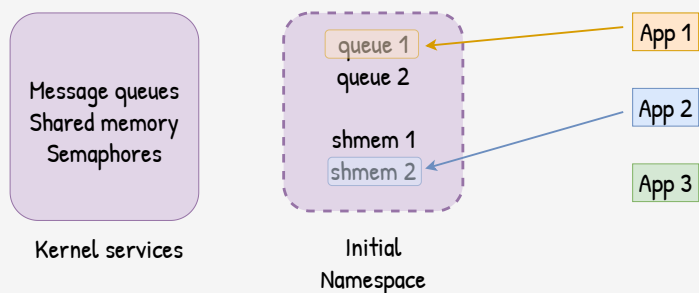
Pero no todo es color de rosa. Al bloquear/falsar syscalls, las apps pueden no funcionar correctamente.

Usando seccomp a secas, es fácil simplificar y filtrar muchas menos syscalls hasta hacer funcionar la app.

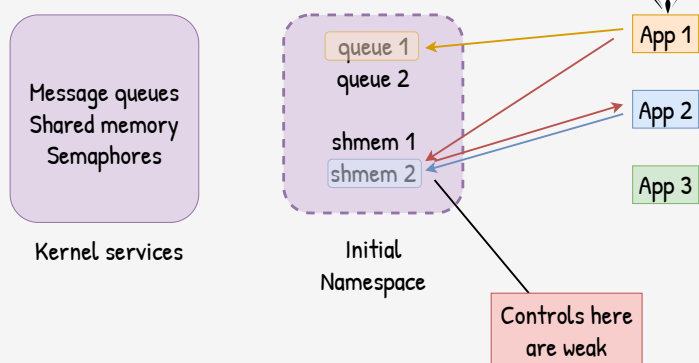
Con un appkernel (como gVisor de Google), es más fácil tener un control completo, a costa de tener una app no trivial corriendo.



IPC



IPC



Si drop permisos y block syscalls es reducir lo q la app puede hacer, el kernel por su parte puede ofrecerle menos desde un principio.

Asi, si una app legítimamente quiere hacer un **connect**, que solo lo haga a travez de las interfaces de red que el sandbox le desee mostrar.

Un browser requiere conectarse a servidores externos pero no debería requerir conectarse a servidores internos escuchando en **lo** (localhost).

El sandbox puede entonces solo mostrarle la placa de wifi y un localhost privado, desconectado del localhost principal.

El kernel ofrece múltiples servicios a las apps de user. En un principio, los recursos de cada servicio se encuentran accesibles en un espacio compartido.

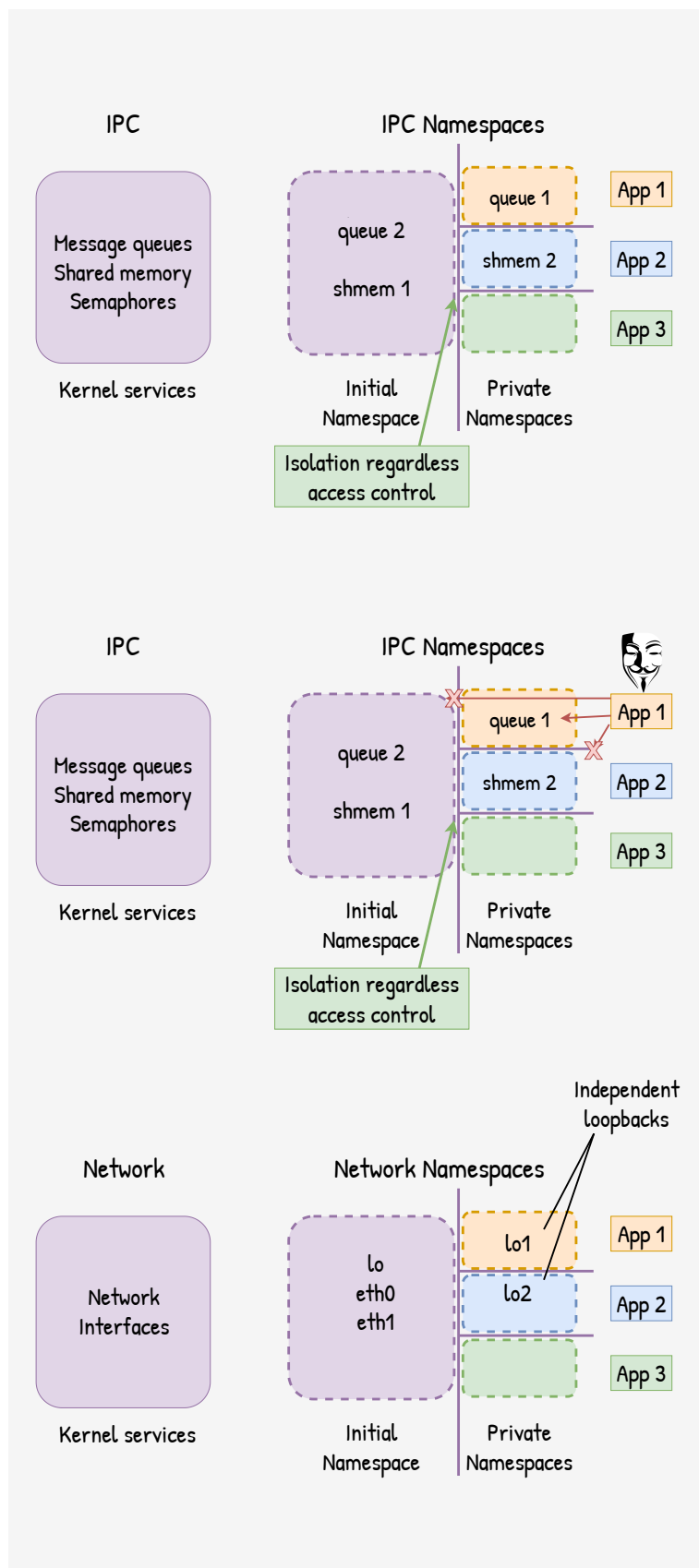
Por ejemplo, inter-process communication (IPC) le permite a las apps intercambiar mensajes entre ellas: queues de mensajes, shared-memories, semaphore sets entre otros.

Pero las apps no tienen uso exclusivo de los IPCs que hayan creado, ni de que otras apps pueden enviar/recibir mensajes de ellos.

Asi, una app maliciosa podría intentar leer/escribir de un IPC de otro proceso, tanto para leakear información o para tomar control de él.

Por default el unico control es basado en users: una app corriendo como un user no puede (salvo mala configuración) acceder al IPC creado por otro user.

En un esquema donde las apps corren bajo el mismo user, este control más que frágil.



El namespace (espacio de nombres) de IPC, limita que IPCs una app puede ver, y por ende, acceder.

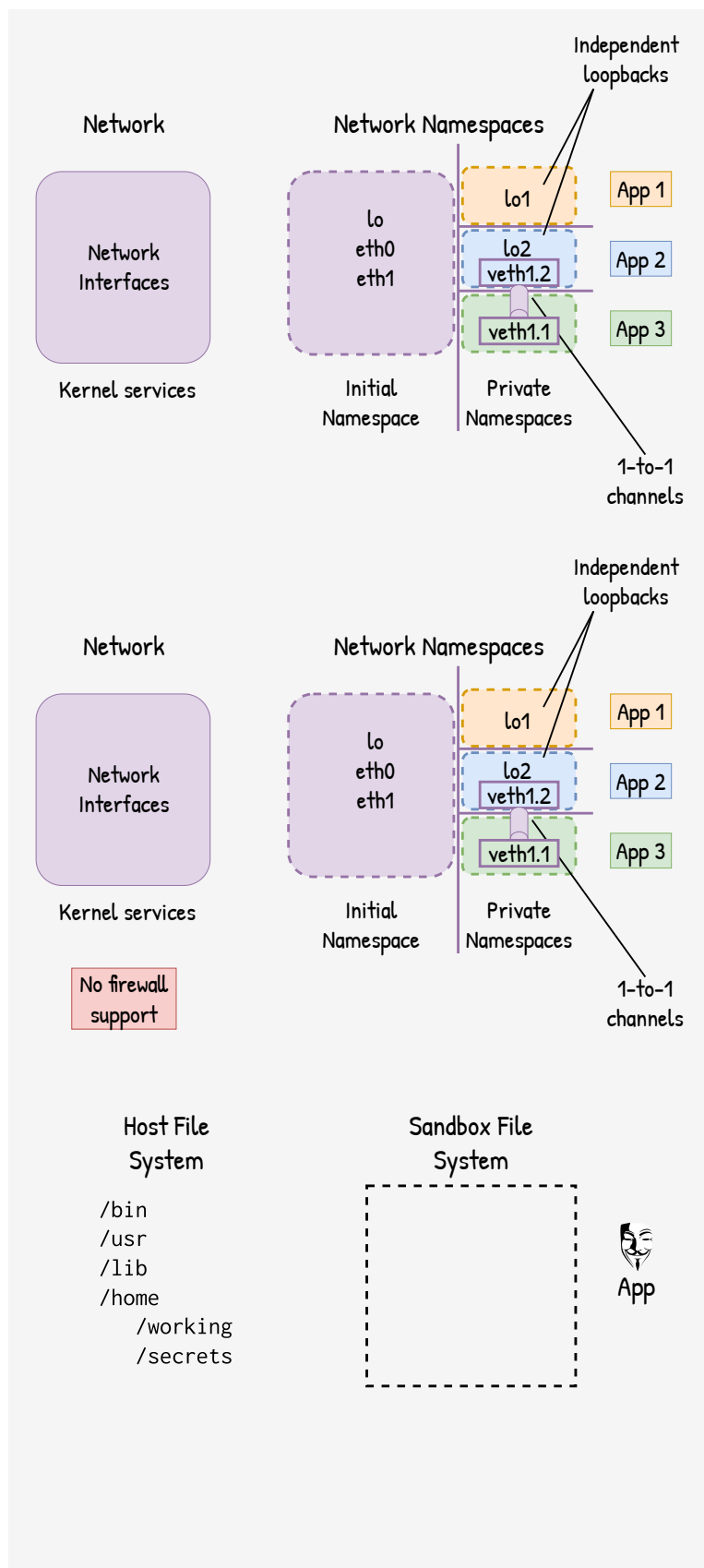
Se deja de estar en un espacio donde todo es compartido y se pasa a un espacio donde una app ve solo lo que necesita. Dentro del namespace una app puede crear todos los IPC que necesite, pero no puede acceder a los de otros namespaces.

Si bien una app puede auto-contenerse (migrar a un namespace), por seguridad es el administrador quien define que namespaces hay y que apps estan en cada uno.

Incluso si las apps corren bajo el mismo user, un adversario no puede salirse de su namespace.

El servicio de IPC no es el único que tiene namespaces. El servicio de users (que users hay en el sistema), el servicio de procesos (que procesos hay), y el de interfaces de red son algunos de los servicios que soportan namespaces, con cada uno teniendo sus particularidades.

Para network, por ejemplo, cada namespace tiene su propia interface the loopback privada y opcionalmente, interfaces de un namespace pueden migrar de un namespace a otro (dictado por el administrador)



Con apps en namespaces de red separadas y sin otra interfaz más que la de loopback, las apps quedan desconectadas de la red (util para evitar que se updateen sin permiso!)

Pero muchas apps estan diseñadas para comunicarse via red, aun localmente. En estos caso es posible create virtual-ethernets: un par de interfaces conectadas entre si, similar a un pipe. Una interfaz esta en un namespace, y la otra en otro y asi hay una forma q las apps pueden comunicarse.

Pero no todo forma parte de un namespace. Firewall, tablas de ruteo, caches son componentes del stack de red pero son compartidos por todos los namespaces de network.

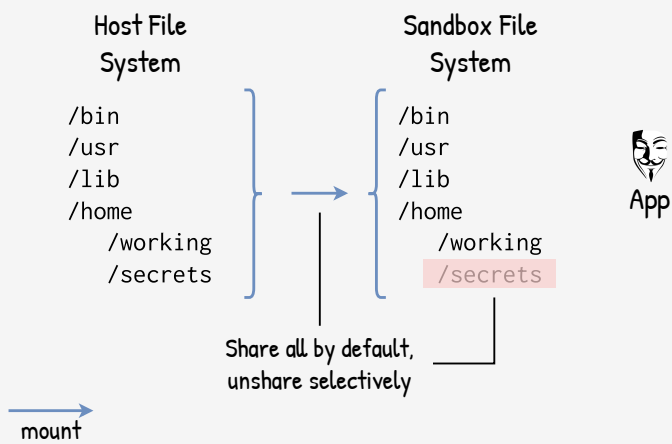
Asi que hay que revisar bien que garantias de separacion cada namespace da y no asumir.

Cuando se quiere separar el file system del host del visto por una app, no es trivial saber que archivos y directorios darle acceso.

A diferencia de los namespaces, donde en general una app puede funcionar perfectamente sin tener acceso a muchas cosas del OS por lo que crear namespaces vacios funcionales es trivial, con los file systems no es asi.

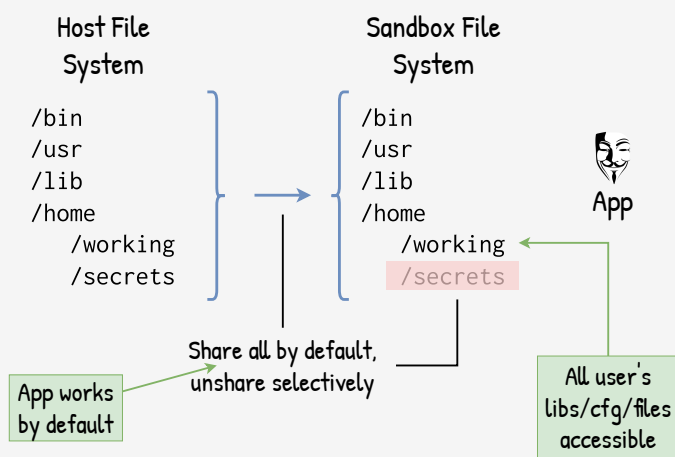
Una app debe poder encontrar el binario de la app (el ejecutable), sus librerías dinámicas (que algunas se cargan solo on-demand), las dependencias de esas librerías, los archivos de configuración y los archivos de data entre otros.

Las apps necesitan de un file systems completo y funcional para funcionar. Como se puede construir uno? Veremos 3 approaches

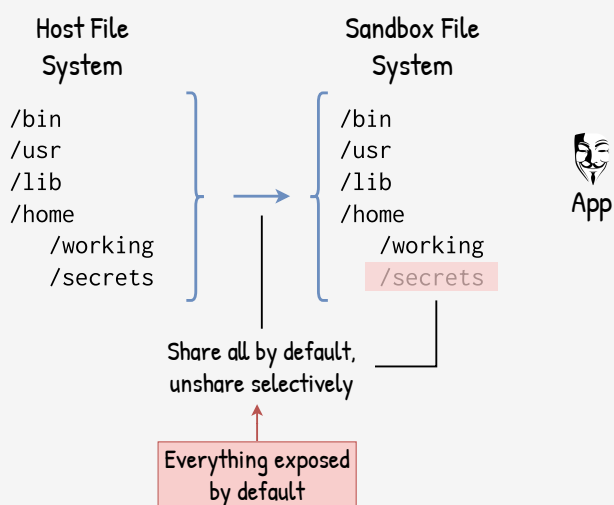


El primero y el más simple es darle acceso a todo: todo el file system del host es montado (mounted) y expuesto a la app.

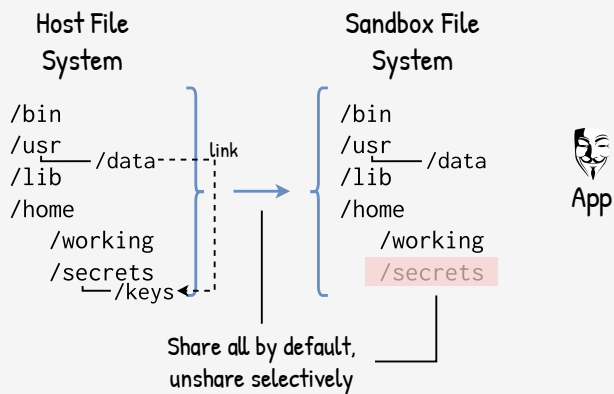
Todo esta compartido por default y es responsabilidad de quien este armando el sandbox de descompartir (unshare) archivos y directorios sensibles.



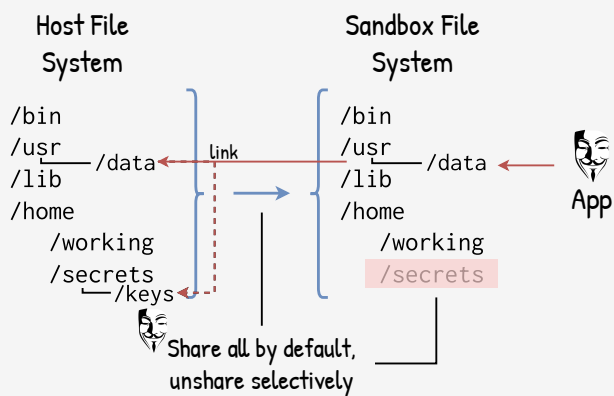
La ventaja inmediata es que la app tiene acceso a todas las libs del host y muy probablemente funcione sin mayores cambios.



La obvia desventaja es que todo esta compartido. Si bien uno puede montar ciertas carpetas y archivos como read-only y descompartir otras, es cuestión de que el administrador cometa un error y deje expuesto algo que no debería.

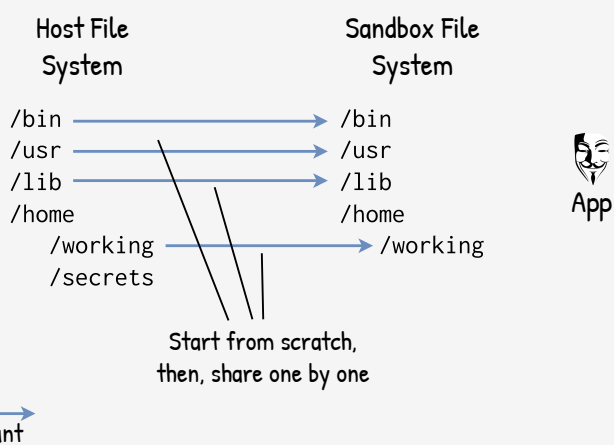


Imaginate un hardlink en `/usr/data` que es un alias de `/home/secrets/keys`.

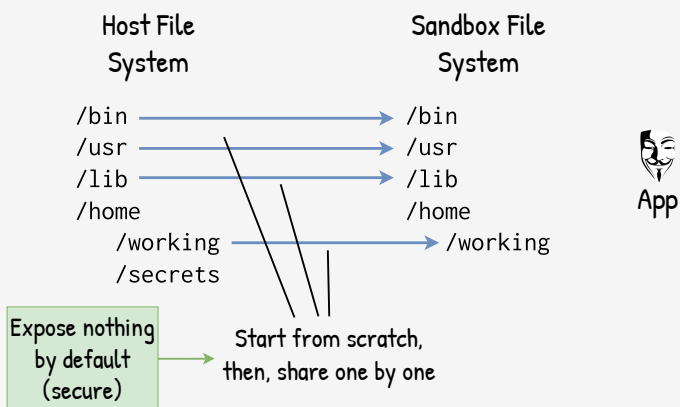


Si lo único que se descompartio (unshare) fue `/home/secrets`, la app aun tiene acceso a `/usr/data` y por ende a `/home/secrets/keys` indirectamente.

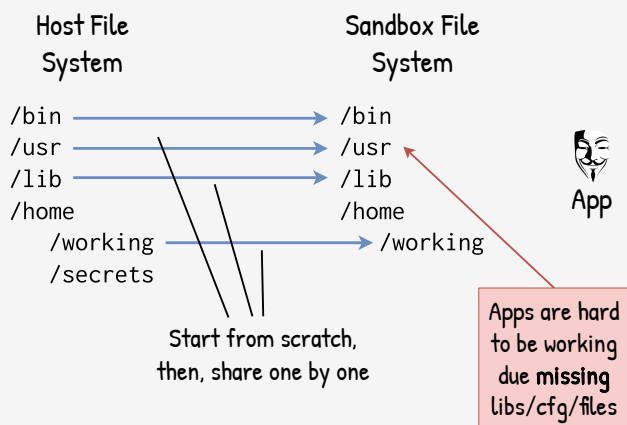
Es muy fácil que algo se te escape.



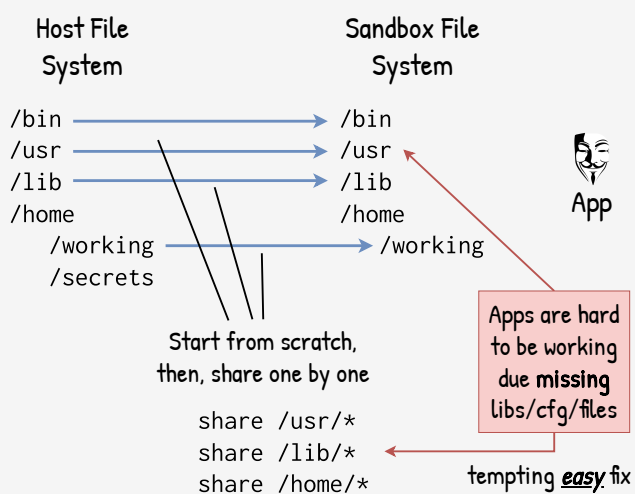
El segundo approach es crear para la app un file system totalmente vacío e ir montando, archivo a archivo, directorio a directorio, lo que se quiera exponer.



Una parte de un escenario donde nada del file system del host está expuesto por lo que es más seguro.

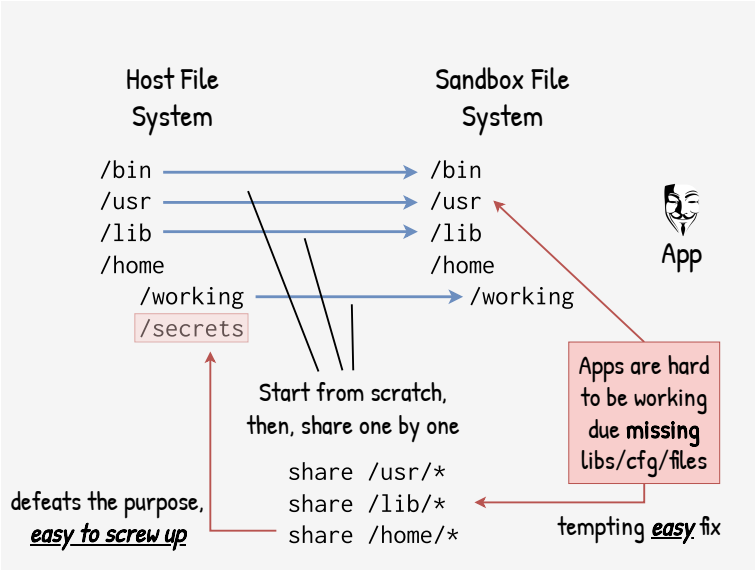


Pero las apps son difíciles de hacerlas funcionar dentro del sandbox por que algo falta (y no siempre es obvio que!)

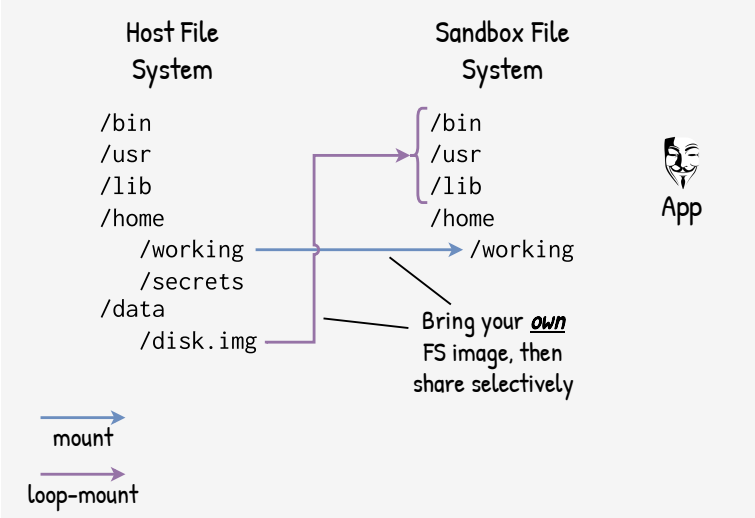


Y para el administrador, puede ser tentador montar (compartir) largas porciones del file system hasta hacer andar la app.

No te olvides que un sistema seguro siempre conspira contra uno que sea fácil de usar y si el usuario se cansa, él va a ser el primero en bypassearte!

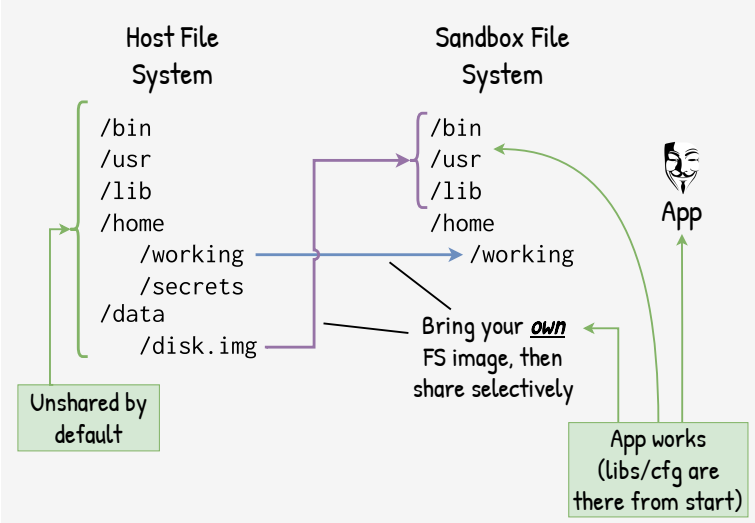


Y el resultado? terminas shareando algo q no deberías.

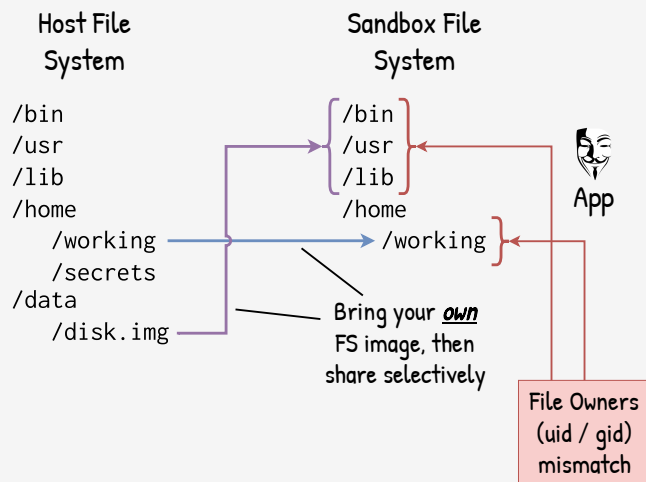


Tercer y último approach: se usa un file system completo y funcional, independiente del host, típicamente guardado en él en un archivo imagen.

Al armar el sandbox, este file system externo se desempaqueta y se monta, y, opcionalmente, se pueden montar archivos y directorios del host adicionalmente.



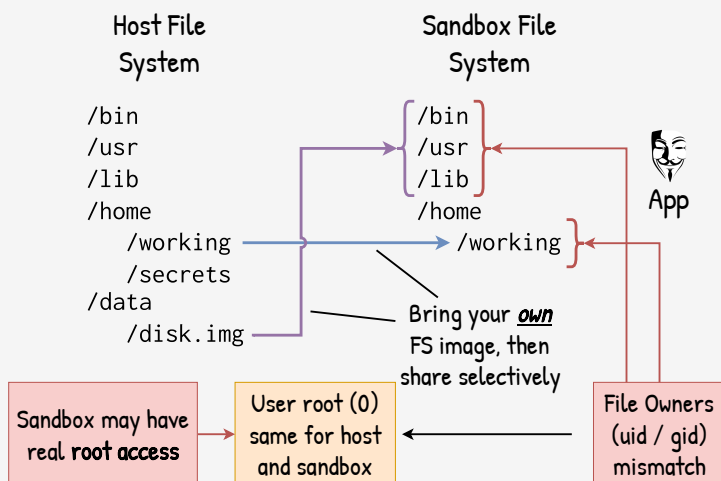
Este esquema no comparte nada por default (es seguro) y a la vez uno parte de un file system que funciona asi q la app debería funcionar (mucho mejor UX)



Pero esto tiene sus issues. Los file systems usan identificadores numéricos (user id, group id) cuyos números no son necesariamente compatibles entre sí.

Así dentro del sandbox, el path `/usr/foo` tiene user id 1001 mientras que `/home/working` tiene user id 1002. En el host, el 1002 es el user Alice; en el sandbox, el 1001 es Alice también, pero 1002 es de Eve.

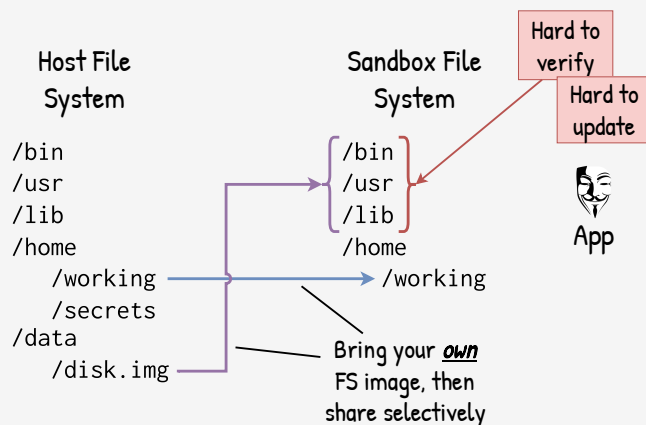
La confusión de ids no solo hace que las cosas no funcionen sino que es un problema de seguridad. No te olvides que el kernel solo controla permisos via estos ids!



El único id que está garantizado funcionar en ambos lados (host y sandbox) es el 0, el id de root.

Así que muchas imágenes (file systems del sandbox) tienen todos sus archivos como root y corren las apps, obviamente como root.

Pero esto malísimo por que la app, aun estando en el sandbox, está corriendo como root y puede escaparse del sandbox!



En kernels modernos hay fixes (más o menos funcionales) de solventar el issue del mismatch de ids.

Pero el approach de usar imágenes pre-armadas de file systems tiene más problemas.

Por ser pre-armadas son difíciles de upgrader; por ser imágenes enteras, son enormes y no es posible verificar que no introduzcan un agujero o bug que le permitan a la app escaparse.

Sandbox Tech	FS	FS Provider	Sbox Cfg
firejail	shared	you	you/3rd
firejail + appimage	img	you/3rd	you/3rd
bubblewrap	unshared	you	you
flatpak	img	3rd	3rd
snap	img	3rd	3rd
docker	img	you/3rd	you



Binary-oriented {
block socket
syscall
drop net
capabilities
apparmor
profile block net

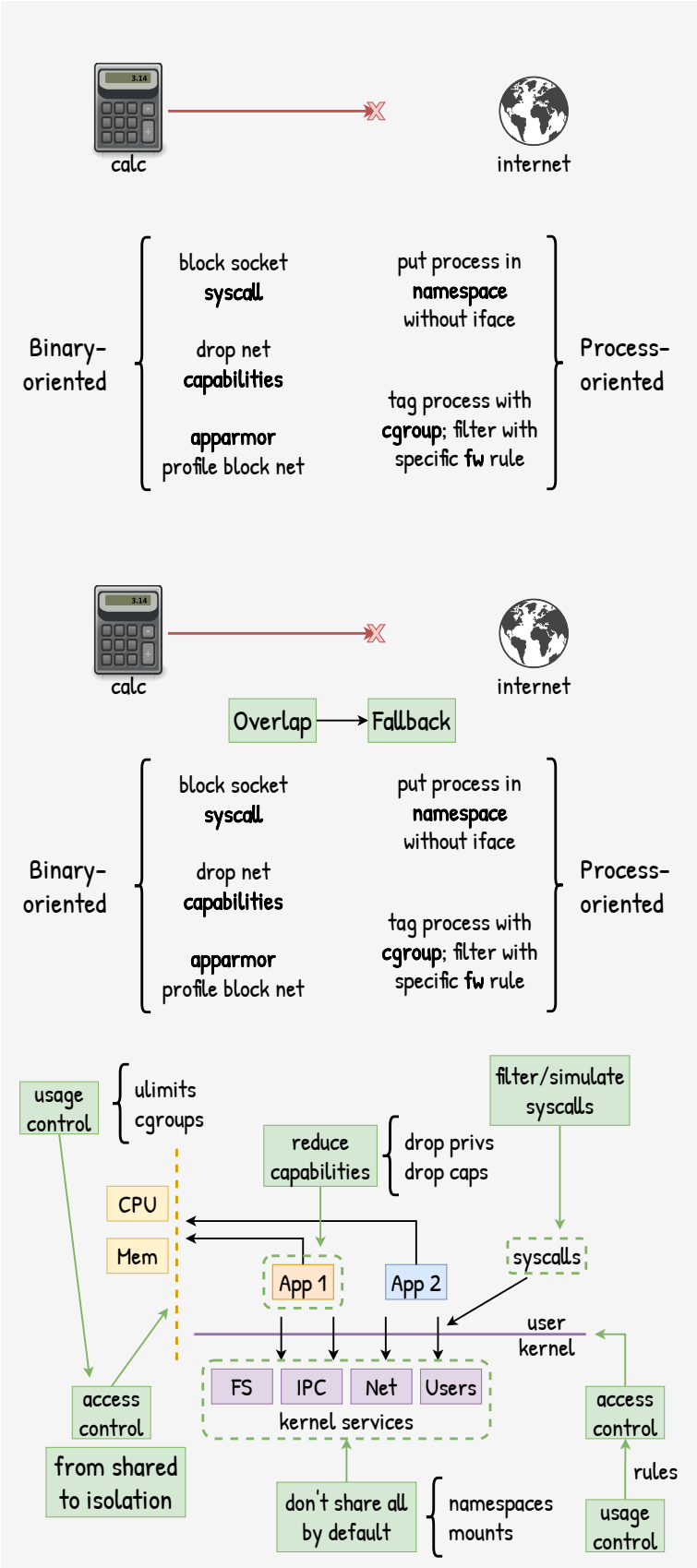
Hay decenas de “orquestradores” que construyen el sandbox con los features del OS. Algunos son más de alto nivel que funcionan con un solo click (snap, flatpak); otros requieren de meterse en los details (docker, bubblewrap, firejail).

Hay siempre una tensión entre usabilidad y seguridad. Así como snap y flatpak son fáciles de usar, uno tiene que confiar 100% en que las imágenes y configuraciones provistas no sean maliciosas.

Ver en detalle como funciona cada sandbox te va a permitir medir bien que tan robusto es. No confíes. Verifica.

El escenario es simple: la app es una calculadora y no queremos q esta se conecte a internet. Como sandboxearla (solo enfocandose en este requerimiento)?

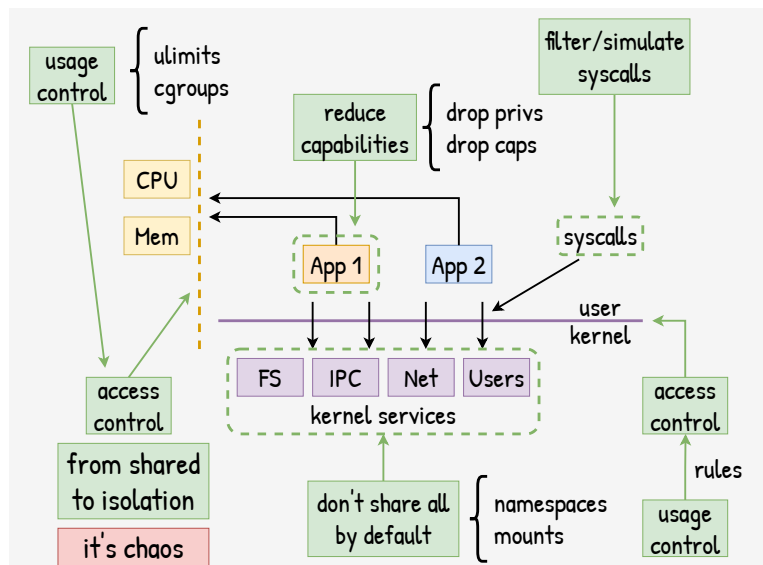
No hay una sola respuesta.



Linux es un universo de tools y techs!

Por que tantas soluciones? Por que hay múltiples formas de resolver un problema y tener varias soluciones permite que si una de las protecciones falle tengamos la otra como fallback/backup.

La construccion del sandbox a nivel OS es tricky. Es tomar algo que por diseño esta compartido e ir bloqueando, separando, reduciendo, aislando de a cachitos.

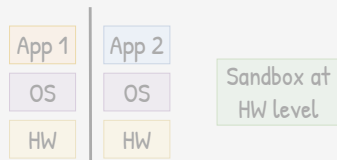
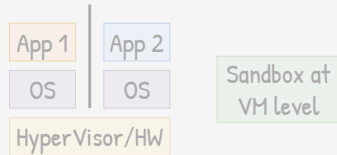
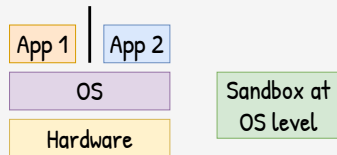


Es un caos. El punto de partida es que todo esta compartido y vasta con olvidarse algo y la app puede escapar del sandbox.

Sin mencionar que las apps interactuan directamente con el kernel quien es el mismo que impone los límites. Un bug en el kernel y es gamo over.

Takeaways

- Efficient
- Great UX
- Customizable



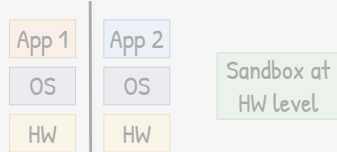
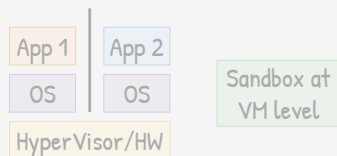
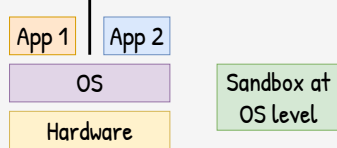
Sandboxes a nivel del OS son eficientes, consumen pocos recursos del OS, son configurables y pueden tener una muy buena UX tanto para el usuario de la app como para quien construye nuevos sandboxes.

Takeaways

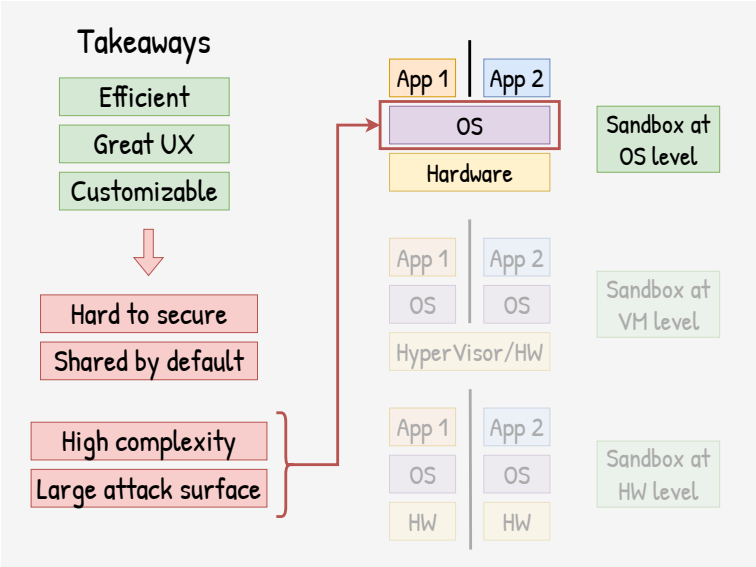
- Efficient
- Great UX
- Customizable



- Hard to secure
- Shared by default



Pero los pros son sus contras. El OS es por default compartido y hacer q cada app viva en su propio universo es muy complicado. Es muy facil que algo se te escape.



Y al final del dia la robustez del sandbox depende de que el kernel haga su trabajo.

Los kernels son enormes, muy complejos y ofrecen muchisimas formas para que las apps interactuen con ellos.

Un bug, y una app puede tomar control del kernel, deshabilitar la seguridad y escapar del sandbox.